Linux Memory Management

Theory, mathematics, and implementation

September 2026

Contents

1 Scope and versions **3**

2 The address space abstraction **4**
2.1 What virtual memory buys . . . 4
2.2 The x86-64 address space . . . 4

3 Page tables **5**
3.1 The radix tree, and why it has that shape . . . 5
3.2 Table memory overhead . . . 6
3.3 PTE format . . . 6
3.4 The TLB . . . 7

4 The physical memory model **7**
4.1 Nodes, zones, and pages . . . 7
4.2 struct page and the memory descriptor problem . . . 8
4.3 Mapping PFNs to pages . . . 9

5 The buddy allocator **9**
5.1 The algorithm . . . 9
5.2 Internal fragmentation . . . 10
5.3 Migratetypes and antifragmentation . . . 10
5.4 Watermarks . . . 11
5.5 The allocation path . . . 11
5.6 GFP flags . . . 12

6 Fragmentation and compaction **13**
6.1 Measuring fragmentation . . . 13
6.2 Compaction . . . 13

7 The object allocator: SLUB **14**
7.1 The problem . . . 14
7.2 Structure . . . 14
7.3 Slab sizing . . . 14
7.4 kmalloc . . . 15
7.5 Debugging . . . 15

8 vmalloc **15**

9 The process address space **16**
9.1 mm_struct and VMAs . . . 16
9.2 The maple tree . . . 16
9.3 Locking . . . 17
9.4 Layout and randomization . . . 17
9.5 Overcommit . . . 17

10 Page fault handling **17**
10.1 The path . . . 17
10.2 Anonymous faults . . . 18
10.3 Copy-on-write . . . 18
10.4 Fault-around . . . 19
10.5 Huge page faults . . . 19

11 The page cache **19**
11.1 Structure . . . 19
11.2 Folios in the page cache . . . 20
11.3 Readahead . . . 20

12 Reverse mapping **20**
12.1 The problem . . . 20
12.2 Object-based rmap . . . 20
12.3 Cost . . . 21

13 Page replacement: the theory **21**
13.1 The optimal policy and why it is unavailable . . . 21
13.2 Stack algorithms and Belady's anomaly . . . 21
13.3 CLOCK and the accessed bit . . . 22

14 Linux's page replacement **22**
14.1 The two-list scheme . . . 22
14.2 Refault distance: measuring what was lost . . . 23
14.3 MGLRU . . . 23

15 Reclaim mechanics **24**
15.1 Who reclaims, and when . . . 24
15.2 The priority loop . . . 24
15.3 Anon versus file balance . . . 24
15.4 The scan itself . . . 25
15.5 Shrinkers . . . 25

16 Swap **26**
16.1 Slots and the swap cache . . . 26
16.2 Writeback of anonymous memory . . . 26

17 Writeback and dirty throttling **26**
17.1 The problem . . . 26
17.2 Limits . . . 27
17.3 The control law . . . 27
17.4 Bandwidth estimation and the rate limit . . . 27
17.5 Per-device and per-cgroup shares . . . 28

18 Huge pages **28**
18.1 THP . . . 28

18.2 hugetlbfs 29

19 NUMA **29**

20 Cgroups and pressure **29**
20.1 memcg 29
20.2 PSI 30

21 The out-of-memory killer **31**

22 Observability **32**

23 Complexity summary **32**

24 Reading path **33**
24.1 Books 33
24.2 Papers 33
24.3 LWN 34

1 Scope and versions

Memory management is the largest and least unified subsystem in the kernel. Unlike the scheduler, it has no single organizing equation; it is a stack of cooperating allocators, a fault handler, a cache, a replacement policy, a writeback control loop, and a set of accounting and pressure mechanisms, each with its own literature.

This document works bottom-up: hardware translation, then the physical page allocator, then the object allocators built on it, then the virtual address space and fault handling, then the page cache and reclaim, then writeback, accounting, and the failure path. Theory and derivations are given where they exist, and every mechanism is tied to the functions that implement it.

At the time of writing the mainline series is 7.2; the 6.18, 6.12 and 6.6 series are long-term. The detailed source references below reflect the 6.x line, which is what the current LTS kernels run and what almost all deployed systems use. Features are dated by the release in which they landed, which does not change. Anything described as "recent" should be checked against your tree.

Everything architecture-specific assumes x86-64 with 4 KiB base pages unless stated; arm64 differences are noted where they matter.

Principal directories and files:

Path	Contents
`mm/page_alloc.c`	Buddy allocator, watermarks, zone fallback
`mm/slub.c`	The object allocator (SLUB)
`mm/vmalloc.c`	Non-contiguous kernel mappings
`mm/memory.c`	Fault handling, page table population, COW
`mm/mmap.c`, `mm/vma.c`	VMA creation, merging, splitting
`mm/filemap.c`, `mm/readahead.c`	Page cache and readahead
`mm/rmap.c`	Reverse mapping
`mm/vmscan.c`, `mm/workingset.c`	Reclaim and refault detection
`mm/page-writeback.c`	Dirty limits and throttling
`mm/compaction.c`, `mm/migrate.c`	Defragmentation
`mm/huge_memory.c`, `mm/hugetlb.c`	Huge pages

Path	Contents
`mm/memcontrol.c`	Cgroup memory accounting
`mm/oom_kill.c`	The out-of-memory killer
`mm/swapfile.c`, `mm/page_io.c`	Swap
`include/linux/mm.h`, `mm_types.h`, `mmzone.h`, `gfp.h`	Core types
`arch/x86/mm/fault.c`	Architecture fault entry
`Documentation/mm/`	In-tree design notes

2 The address space abstraction

2.1 What virtual memory buys

Virtual memory is usually taught as a way to run programs larger than RAM. That is the least interesting thing it does, and on modern systems it is close to irrelevant. The properties that actually matter:

- **Isolation.** Two processes can hold the same virtual address without seeing each other's data. Protection is enforced by hardware on every access, at zero marginal cost.
- **Relocation.** A program can be linked at a fixed address and loaded anywhere physical. Address space layout randomization builds on this.
- **Sparseness.** A 47-bit address space can be almost entirely unmapped. Cost is proportional to what is used, not what is addressable.
- **Overcommit and lazy allocation.** A mapping can be promised without being backed, and materialized on first touch. Most `malloc()` memory is never touched.
- **Sharing.** One physical page can appear in many address spaces. Shared libraries, `fork()`, the page cache, and KSM all depend on it.
- **Indirection for policy.** Because every access goes through a translation the kernel controls, the kernel can interpose: copy-on-write, demand paging, swapping, migration, dirty tracking, NUMA balancing.

The cost is a translation on every memory access, which is why the TLB exists and why so much of this document is ultimately about TLB and cache economics.

2.2 The x86-64 address space

x86-64 does not use all 64 bits. With 4-level paging, 48 bits are translated and addresses must be **canonical**: bits 48–63 must all equal bit 47. This splits the space into two halves separated by a large non-canonical hole, which conveniently gives userspace the low half and the kernel the high half with a hardware-enforced gap between them.

Region	Range	Size
User space	`0000000000000000`–`00007fffffffffff`	128 TiB
Non-canonical hole	—	16 EiB minus 256 TiB
Direct map of all RAM	`ffff888000000000`–`ffffc87fffffffff`	64 TiB
`vmalloc` / `ioremap`	`ffffc90000000000`–`ffffe8ffffffffff`	32 TiB

Region	Range	Size
`vmemmap` (`struct page` array)	`ffffea0000000000`–`ffffeaffffffffff`	1 TiB
Kernel text	`ffffffff80000000`–`ffffffff9fffffff`	512 MiB

With 5-level paging (`CONFIG_X86_5LEVEL`, Ice Lake and later, enabled at boot if the hardware supports it) the translated width becomes 57 bits and each half grows to 128 PiB. The full map is in `Documentation/arch/x86/x86_64/mm.rst`.

The **direct map** (also called the linear map, `page_offset_base`) is the single most important kernel mapping: every physical page of RAM is mapped at a fixed offset, so conversion is arithmetic rather than a lookup:

$$\texttt{\_\_va}(p) = p + \texttt{PAGE\_OFFSET}, \qquad \texttt{\_\_pa}(v) = v - \texttt{PAGE\_OFFSET}.$$

This is why `kmalloc()` memory is physically contiguous and DMA-capable while `vmalloc()` memory is not, and it is why 32-bit kernels needed highmem: a 1 GiB kernel half cannot directly map 4 GiB of RAM. Highmem was removed for 64-bit long ago and is now effectively a historical curiosity.

3 Page tables

3.1 The radix tree, and why it has that shape

A page table is a lookup from virtual page number to physical frame number. A flat array would need one entry per virtual page: with 4 KiB pages and 8-byte entries, a 48-bit space needs 2^{36} entries, or 512 GiB of table per process. Unusable.

The fix is a **radix tree** (a trie on address bits), which costs memory only for populated subtrees. The branching factor falls out of the page size: one 4 KiB page holding 8-byte entries holds

$$\frac{4096}{8} = 512 = 2^9$$

entries, so each level consumes exactly 9 address bits. With 4 levels,

$$9 \times 4 + 12 = 48 \text{ bits},$$

and with 5 levels, $9 \times 5 + 12 = 57$ bits. The architecture's address width is not a design choice made in isolation; it is what the page size and entry size produce.

Linux names the levels, from the top:

Level	Name	Bits	Covers
1	PGD (page global directory)	47:39	512 GiB
2	P4D	38:30 (4-level: folded)	—
3	PUD (page upper directory)	38:30	1 GiB
4	PMD (page middle directory)	29:21	2 MiB

Level	Name	Bits	Covers
5	PTE (page table entry)	20:12	4 KiB

P4D was added in 4.11 to prepare for 5-level paging. On 4-level hardware it is **folded**: `p4d_offset()` is an identity function and the level costs nothing. This folding trick (`include/asm-generic/pgtable-nop4d.h` and friends) lets generic mm code always write five levels of walk while architectures with fewer levels compile them away.

A walk in generic code looks like:

```
pgd = pgd_offset(mm, addr);
p4d = p4d_offset(pgd, addr);
pud = pud_offset(p4d, addr);
pmd = pmd_offset(pud, addr);
pte = pte_offset_map(pmd, addr);
```

with `*_none()`, `*_bad()`, and `*_present()` predicates at each step. `mm/pagewalk.c` provides a generic visitor (`walk_page_range()`) used by everything from `/proc/PID/smaps` to `mprotect()`.

3.2 Table memory overhead

For a densely populated mapping the dominant cost is the leaf level: one 8-byte PTE per 4 KiB page, so

$$\text{overhead} = \frac{8}{4096} = \frac{1}{512} \approx 0.195\%,$$

plus a geometric tail of $1/512$ per additional level, giving

$$\frac{1}{512}\left(1 + \frac{1}{512} + \frac{1}{512^2} + \cdots\right) = \frac{1}{511} \approx 0.196\%.$$

Negligible for one process. It stops being negligible when the same file is mapped by thousands of processes, since each gets its own leaf tables: 1000 processes mapping 1 GiB each pay 2 GiB in page tables to map 1 GiB of data. This is the motivation for `mm/mmap.c` page-table sharing for hugetlb (`huge_pmd_share()`) and for the more general **mshare** work. Page table bytes are tracked per-mm (`mm_pgtables_bytes()`) and appear in the OOM badness score for exactly this reason.

3.3 PTE format

An x86-64 PTE is 64 bits:

Bit(s)	Name	Meaning
0	P	Present. If clear, the rest is software-defined.
1	R/W	Writable
2	U/S	User-accessible
3, 4	PWT, PCD	Cache policy (with PAT)
5	A	Accessed — set by hardware on reference
6	D	Dirty — set by hardware on write
7	PSE/PAT	At PMD/PUD level: this is a huge page leaf

Bit(s)	Name	Meaning
8	G	Global (not flushed on CR3 write)
9–11, 52–58	—	Available to software
12–51	PFN	Physical frame number
59–62	PKEY	Protection key
63	NX	No-execute

Two of these carry most of the mm subsystem's weight. The **accessed** bit is the hardware's only contribution to reference tracking, and the entire page replacement policy is built on sampling and clearing it. The **dirty** bit drives writeback. Both are set by hardware and cleared by software, which makes clearing them a TLB-shootdown-bearing operation and therefore expensive at scale — a fact that shapes the design of reclaim.

When bit 0 is clear the entry is entirely software-defined, and Linux uses this to encode **swap entries**: a type field selecting the swap device and an offset within it, plus migration entries, hardware-poison entries, and (with `CONFIG_DEVICE_PRIVATE`) device-private entries. See `swp_entry_t` and the `include/linux/swapops.h` accessors. A non-present PTE is not necessarily an empty PTE, and code that assumes otherwise is a recurring bug class.

3.4 The TLB

Translation is cached in the TLB. The quantity that matters is **reach**:

$$\text{reach} = (\text{entries}) \times (\text{page size}).$$

A typical x86-64 core has on the order of 1500–3000 second-level TLB entries. At 4 KiB that is roughly

$$2048 \times 4\ \text{KiB} = 8\ \text{MiB},$$

which is smaller than the L3 cache and vastly smaller than working sets of interest. This single number explains huge pages: the same 2048 entries at 2 MiB reach 4 GiB, a 512-fold improvement, and it explains why database and JVM workloads care so much about THP.

TLB invalidation is the expensive part. `INVLPG` invalidates one entry on one CPU; there is no cross-CPU broadcast on x86, so the kernel sends IPIs (**TLB shootdown**, `arch/x86/mm/tlb.c`, `flush_tlb_mm_range()`). Cost is $O(\text{CPUs in } \texttt{mm\_cpumask})$, and the kernel batches aggressively (`tlb_gather_mmu()` / `tlb_finish_mmu()` in `mm/mmu_gather.c`) and falls back to a full flush when the range is large, since a full flush is a constant cost and per-page invalidation is linear. The crossover is `tlb_single_page_flush_ceiling`, default 33.

PCID (process context identifiers) lets the TLB retain entries for multiple address spaces across `CR3` writes, which is what made KPTI (page table isolation for Meltdown) survivable: without PCID, KPTI's two-page-table-per-process design flushes the TLB on every syscall.

4 The physical memory model

4.1 Nodes, zones, and pages

Physical memory is described by a three-level hierarchy in `include/linux/mmzone.h`:

- **Node** (`struct pglist_data`, conventionally `pgdat`) — one per NUMA node. Owns the LRU lists, the reclaim state, and `kswapd`.
- **Zone** (`struct zone`) — a range within a node with a distinct allocation constraint.
- **Page** (`struct page`) — one per physical frame.

Zones exist to encode addressing constraints, not policy:

Zone	Purpose
`ZONE_DMA`	Below 16 MiB. Ancient ISA devices.
`ZONE_DMA32`	Below 4 GiB. Devices with 32-bit DMA masks.
`ZONE_NORMAL`	Everything else. The bulk of memory.
`ZONE_MOVABLE`	Pseudo-zone holding only migratable pages; enables hotplug and large contiguous allocations.
`ZONE_DEVICE`	Not RAM. Persistent memory, device memory (`CONFIG_ZONE_DEVICE`).

An allocation names the highest acceptable zone via GFP flags, and the allocator walks a **zonelist** downward from there. The zonelist is built per node in `build_zonelists()` and encodes NUMA preference: the local node's zones first, then remote nodes ordered by distance from the SLIT table.

The `lowmem_reserve` mechanism prevents a subtle starvation: an unconstrained `ZONE_NORMAL` allocation could exhaust `ZONE_DMA32` through fallback, after which a genuinely 32-bit-constrained device allocation has nowhere to go. Each zone therefore reserves a fraction of itself against allocations that could have been satisfied higher up, sized by `sysctl_lowmem_reserve_ratio` (default `256, 256, 32, 0, 0`), and visible per zone in `/proc/zoneinfo`.

4.2 struct page and the memory descriptor problem

`struct page` is one of the most contended structures in the kernel. There is one per 4 KiB frame, so its size is a direct tax on all RAM:

$$\frac{64\text{ bytes}}{4096\text{ bytes}} = 1.5625\%.$$

On a 1 TiB machine that is 16 GiB spent describing memory. Every byte added costs 0.024% of all RAM, which is why the structure is a dense union of overlapping interpretations — the same words mean different things depending on whether the page is anonymous, page cache, slab, a page table, or free.

Two long-running efforts address this:

Folios (Matthew Wilcox, 5.16–5.17 onward). A `struct folio` is the head page of a possibly-compound allocation, with the order recorded. Before folios, code that received a `struct page *` could not tell whether it had a head page, a tail page, or a lone page, and the resulting confusion was a steady source of bugs; APIs also silently assumed 4 KiB. Folios make the size explicit in the type and let the page cache manage large blocks with one descriptor and one refcount instead of 2^n. Much of `mm/filemap.c`, `mm/rmap.c` and `mm/vmscan.c` has been converted.

Memory descriptors ("memdesc"). The longer-term plan: shrink `struct page` to a single word that mostly acts as a tagged pointer to a type-specific descriptor (`struct folio`, `struct slab`,

`struct ptdesc`), allocated only for pages that need one. `struct slab` and `struct ptdesc` have already been split out of `struct page`.

4.3 Mapping PFNs to pages

`SPARSEMEM` (with `SPARSEMEM_VMEMMAP` on 64-bit) divides the physical space into **sections** (128 MiB on x86-64, `SECTION_SIZE_BITS = 27`) and allocates `struct page` arrays per populated section. With vmemmap the arrays are placed in a dedicated virtual region so the array appears contiguous even when physical memory is not, which makes the conversions arithmetic again:

```
#define __pfn_to_page(pfn)  (vmemmap + (pfn))
#define __page_to_pfn(page) ((unsigned long)((page) - vmemmap))
```

This is why `pfn_to_page()` is free on x86-64 despite arbitrary physical memory holes, and it is what `CONFIG_SPARSEMEM_VMEMMAP` is really buying. Memory hotplug adds and removes whole sections; `mm/sparse.c` and `mm/memory_hotplug.c` manage them. `mm/sparse-vmemmap.c` additionally supports mapping the vmemmap pages of a hugetlb page onto a single shared page (`hugetlb_free_vmemmap`), recovering most of the 1.56% for hugetlb memory.

5 The buddy allocator

5.1 The algorithm

The page allocator (`mm/page_alloc.c`) is a **binary buddy system**, from Knuth via Knowlton (1965). Free memory is kept as blocks of 2^k contiguous pages for $k = 0 \ldots$ `MAX_PAGE_ORDER`, with one free list per order per migratetype per zone:

```
struct free_area {
        struct list_head        free_list[MIGRATE_TYPES];
        unsigned long           nr_free;
};
struct zone {
        ...
        struct free_area        free_area[NR_PAGE_ORDERS];
};
```

On x86-64 `MAX_PAGE_ORDER` is 10, so the largest block the allocator will hand out is $2^{10} \times 4$ KiB $=$ 4 MiB. (Before 6.8 this constant was `MAX_ORDER` and was exclusive, i.e. 11; the rename in 6.8 made it inclusive and fixed a decade of off-by-one confusion. Check which convention your tree uses.)

Allocation of order k: take a block from list k if non-empty; otherwise find the smallest $j > k$ with a free block, remove it, and split repeatedly, returning each unused half to its list. Splitting is $O($`MAX_PAGE_ORDER`$)$, so bounded by 10.

Freeing of order k at frame p: compute the buddy and coalesce if it is free and of the same order, then repeat upward. The buddy is a single XOR:

$$\texttt{buddy}(p, k) \;=\; p \oplus 2^k.$$

```
static inline unsigned long __find_buddy_pfn(unsigned long page_pfn,
                                             unsigned int order)
{
        return page_pfn ^ (1 << order);
}
```

This works because a block of order k is 2^k-aligned, so its address has k low zero bits; flipping bit k names the adjacent block that would merge with it. It is the whole reason buddy systems are fast: coalescing needs no search, no free-list scan, and no boundary tags — just an XOR and a check of the neighbour's order and state, which live in `page->private` and the `PageBuddy` flag.

Both operations are $O(\log_2 \texttt{MAX\_PAGE\_ORDER})$, which is at most 10 steps.

5.2 Internal fragmentation

Rounding every request to a power of two wastes space. If request sizes are uniform over $(2^{k-1}, 2^k]$, the expected allocation is 2^k and the expected request is $\frac{3}{4}2^k$, so

$$\mathbb{E}\left[\frac{\text{waste}}{\text{allocated}}\right] = 1 - \frac{\frac{3}{4}2^k}{2^k} = \frac{1}{4}.$$

25% expected internal waste. That is a great deal to give up, and it is precisely why the buddy allocator is *not* the general-purpose allocator: it hands out pages, and a separate object allocator (Section 8) subdivides them. In practice the overwhelming majority of buddy allocations are order 0, where the waste is zero by definition, and higher orders are requested by code that genuinely wants a power-of-two block.

5.3 Migratetypes and antifragmentation

Buddy coalescing only works if neighbours are simultaneously free. A single unmovable 4 KiB allocation sitting in the middle of an otherwise free 4 MiB region prevents that region from ever coalescing — **external fragmentation**, and the reason a machine with gigabytes free can fail an order-9 allocation.

Linux attacks this by segregating allocations by *mobility* (Mel Gorman, 2.6.24). Each **pageblock** (order-9, i.e. 2 MiB on x86-64) carries a migratetype:

Type	Meaning
`MIGRATE_UNMOVABLE`	Kernel data. Cannot be relocated.
`MIGRATE_MOVABLE`	Anonymous and page-cache pages. Relocatable via `rmap`.
`MIGRATE_RECLAIMABLE`	Slab caches with shrinkers; freeable under pressure.
`MIGRATE_CMA`	Contiguous Memory Allocator reserve.
`MIGRATE_ISOLATE`	Temporarily off-limits (hotplug, CMA, compaction).

Allocations draw from the list matching their GFP mobility (`GFP_MOVABLE` and friends), so unmovable kernel objects cluster together and leave large movable regions intact. When a type is exhausted, `__rmqueue_fallback()` steals from another, preferring to steal the *largest* available block and, when the steal is large enough, converting the whole pageblock's type so the damage stays contained rather than scattering.

This is a heuristic with no proof behind it, but it is enormously effective in practice: it is the difference between THP allocation succeeding after days of uptime and failing after hours.

5.4 Watermarks

Every zone has three watermarks that gate allocation and drive reclaim:

$$\texttt{min} < \texttt{low} < \texttt{high}.$$

- Above `high`: `kswapd` sleeps.
- Below `low`: `kswapd` wakes and reclaims in the background; allocations still succeed.
- Below `min`: allocations enter **direct reclaim** — the allocating task does the reclaim work itself, synchronously. Only `PF_MEMALLOC` contexts and `__GFP_HIGH`/`__GFP_ATOMIC` allocations may dip below.

The base is `min_free_kbytes`, whose default is chosen in `init_per_zone_wmark_min()` to scale as a **square root** of memory:

```
lowmem_kbytes = nr_free_buffer_pages() * (PAGE_SIZE >> 10);
new_min_free_kbytes = int_sqrt(lowmem_kbytes * 16);
```

That is,

$$\texttt{min\_free\_kbytes} = 4\sqrt{\texttt{lowmem\_kbytes}}, \qquad \text{clamped to } [128,\ 65536].$$

Square-root scaling is the right shape: the reserve must cover the burst of allocation that can occur between crossing a watermark and reclaim producing results, which grows with system throughput rather than with system size. A 16 GiB machine gets 16 MiB; the 64 MiB ceiling is reached at exactly 256 GiB, and above that the default stops growing — which is why large machines frequently need it raised by hand.

The min is divided among zones in proportion to managed pages, and the upper two are then derived in `__setup_per_zone_wmarks()`:

$$\Delta = \max\left(\frac{\texttt{min}}{4},\ \texttt{managed\_pages} \cdot \frac{\texttt{watermark\_scale\_factor}}{10000}\right),$$
$$\texttt{low} = \texttt{min} + \Delta, \qquad \texttt{high} = \texttt{min} + 2\Delta.$$

`watermark_scale_factor` defaults to 10, i.e. 0.1% of the zone. Raising it widens the band between `low` and `high`, giving `kswapd` more room to work asynchronously before anyone hits direct reclaim; this is the standard tuning knob for latency-sensitive workloads that see allocation stalls.

The check itself, `zone_watermark_ok()`, is more than a comparison against free pages. It subtracts `lowmem_reserve`, excludes CMA pages for non-CMA requests, and for order > 0 additionally verifies that a block of sufficient order actually exists — free pages are necessary but not sufficient for a high-order allocation.

5.5 The allocation path

```
alloc_pages()
  -> __alloc_pages()
       get_page_from_freelist()            /* fast path: watermarks OK */
       __alloc_pages_slowpath()
            wake_all_kswapds()
            get_page_from_freelist(ALLOC_WMARK_MIN)
            __alloc_pages_direct_compact()  /* high order only */
            __alloc_pages_direct_reclaim()
```

```
__alloc_pages_may_oom()
/* retry loop with should_reclaim_retry() */
```

The fast path is deliberately short and lock-light. Order-0 allocations mostly never reach the zone lock at all: each CPU keeps a **per-CPU pageset** (`struct per_cpu_pages`, the "pcp lists") of free order-0 (and, since 5.x, small high-order) pages, refilled in batches under the zone lock. This is the single most important scalability property of the allocator, since order-0 is the overwhelming majority of traffic. Batch size scales with zone size and is capped; drained by `drain_all_pages()` when pressure demands accuracy.

The slow path is a retry loop with escalating aggression, and its termination condition (`should_reclaim_retry()`) is genuinely delicate: give up too early and allocations fail spuriously, give up too late and the machine livelocks in reclaim instead of invoking the OOM killer. Historically this loop was `__GFP_NOFAIL`-adjacent for small orders — allocations of order ≤ 3 effectively looped forever — and much of the modern code exists to make failure and OOM invocation deterministic.

5.6 GFP flags

The `gfp_t` argument encodes three orthogonal things: which zones are acceptable, what the allocator is permitted to do to satisfy the request, and how the page will be used. From `include/linux/gfp_types.h`:

Flag	Effect
`__GFP_DIRECT_RECLAIM`	May block and reclaim in this context
`__GFP_KSWAPD_RECLAIM`	May wake `kswapd`
`__GFP_IO`, `__GFP_FS`	May start I/O; may recurse into filesystems
`__GFP_HIGH`	May use emergency reserves
`__GFP_NOWARN`, `__GFP_RETRY_MAYFAIL`, `__GFP_NOFAIL`	Failure behaviour
`__GFP_ZERO`, `__GFP_COMP`, `__GFP_ACCOUNT`	Post-processing, compound, memcg-charged

The familiar composites:

Composite	Definition	Use
`GFP_KERNEL`	reclaim + IO + FS	Normal sleepable context
`GFP_NOFS`	reclaim + IO	Inside a filesystem; must not recurse
`GFP_NOIO`	reclaim only	Inside block layer
`GFP_ATOMIC`	`__GFP_HIGH`, kswapd wake, no direct reclaim	Interrupt context
`GFP_USER`, `GFP_HIGHUSER_MOVABLE`	+ zone and mobility hints	Userspace pages

The `NOFS`/`NOIO` distinction encodes a **deadlock avoidance** invariant, not a performance preference. If a filesystem allocates memory while holding a lock, and that allocation enters reclaim, and reclaim tries to write back a dirty page belonging to the same filesystem, and writeback needs that lock, the system deadlocks. The scoped API (`memalloc_nofs_save()` / `memalloc_nofs_restore()`,

and the `noreclaim/nofs/noio` family in `include/linux/sched/mm.h`) is the modern way to express this, since it sets the constraint for a region of code rather than requiring every allocation site inside to remember the right flag.

6 Fragmentation and compaction

6.1 Measuring fragmentation

The kernel needs to distinguish two failure modes for a high-order allocation: there is not enough free memory (reclaim will help), or there is enough free memory but it is scattered (compaction will help, reclaim will not). `__fragmentation_index()` in `mm/vmstat.c` produces a number in $[0, 1000]$ separating them:

$$F(k) \;=\; 1000 - \frac{1000 + 1000 \cdot \dfrac{\texttt{free\_pages}}{2^k}}{\texttt{free\_blocks\_total}}.$$

The reading is:

- $F \to 0$: few free pages overall. The allocation fails for lack of memory. **Reclaim.**
- $F \to 1000$: many free pages, spread across many small blocks. The allocation fails for lack of contiguity. **Compact.**

The numerator counts how many blocks of the requested order the free memory *could* form; the denominator counts how many blocks it actually occupies. The index is only computed when the allocation would fail — if a suitable block exists the function returns -1000 and nothing else runs. `compaction_suitable()` compares against `extfrag_threshold` (default 500) to decide whether compaction is worth attempting. The raw inputs are exported in `/proc/pagetypeinfo` and `/sys/kernel/debug/extfrag/`.

6.2 Compaction

`mm/compaction.c` defragments by moving pages, using two scanners that walk toward each other within a zone:

- The **migration scanner** starts at the low end and collects movable pages.
- The **free scanner** starts at the high end and collects free pages.

Pages from the first are migrated into pages from the second, sweeping used pages downward and accumulating free space at the top. When the scanners meet, the zone has been compacted once.

Only `MIGRATE_MOVABLE` pages can be moved, and moving one requires finding and updating every PTE that maps it — that is what reverse mapping (Section 13) is for. Anonymous pages and page cache pages qualify; slab objects, page tables and most kernel allocations do not, which is the deeper reason antifragmentation grouping matters.

Compaction runs in three contexts: synchronously from the allocator's slow path (`__alloc_pages_direct_compact()`), asynchronously via `kcompactd` (one per node), and on demand from `/proc/sys/vm/compact_memory`. The direct case is latency-critical — a task is blocked in a page fault waiting for it — so it uses `MIGRATE_ASYNC`, which refuses to block on page locks or writeback and gives up quickly. `kcompactd` uses the synchronous mode. Deferral logic (`defer_compaction()`, exponential backoff in `zone->compact_considered`) prevents repeated futile attempts from burning CPU, a real problem in the early THP days when compaction stalls were the single largest source of THP's bad reputation.

7 The object allocator: SLUB

7.1 The problem

The buddy allocator's smallest unit is a page. The kernel allocates enormously more `struct dentry` (about 192 bytes) and `struct inode` objects than it does pages, and giving each a full page would waste 95% of memory and destroy cache locality. A **slab allocator** sits on top of the page allocator and subdivides pages into same-sized objects.

Jeff Bonwick's 1994 slab design (from SunOS) contributed three ideas: per-object-type caches so that objects of a type are packed together and share cache lines; object *constructors* so that a freed object can be reused without reinitializing invariant fields; and per-CPU caching of free objects to avoid a shared lock on the hot path.

Linux has carried three implementations. SLOB (tiny systems) was removed in 6.4; the original SLAB was removed in 6.8. **SLUB** (Christoph Lameter, 2.6.22) is now the only one, and `mm/slub.c` is the file to read.

7.2 Structure

SLUB keeps per-cache, per-CPU state and is deliberately simpler than SLAB, which maintained per-node array caches with elaborate batching:

```
struct kmem_cache_cpu {
        void                    **freelist;    /* next available object */
        unsigned long           tid;           /* for cmpxchg_double */
        struct slab             *slab;         /* the active slab */
        struct slab             *partial;      /* partially full, per cpu */
};
```

The fast path is remarkable for what it does not do:

```
/* conceptually, kmem_cache_alloc(): */
object = c->freelist;
if (likely(object && node_match(...))) {
        c->freelist = get_freepointer(s, object);
        /* single cmpxchg_double on (freelist, tid) makes this atomic */
}
```

Free objects are threaded into a **linked list stored inside the free objects themselves** — the "free pointer" occupies bytes of the object that are meaningless while it is free. So there is no separate metadata array, no bitmap, and no per-object header in the common case. Allocation is a pointer dereference and a store; free is a store and a store. Both are lockless, using `cmpxchg_double` on the (freelist, tid) pair to detect preemption.

When the per-CPU freelist empties, `___slab_alloc()` takes the per-CPU partial list, then the node partial list, and finally asks the page allocator for a fresh slab.

7.3 Slab sizing

For an object of size s and a slab of 2^k pages, the object count and waste are

$$n = \left\lfloor \frac{2^k \cdot \texttt{PAGE\_SIZE}}{s} \right\rfloor, \qquad \text{waste} = 1 - \frac{n s}{2^k \cdot \texttt{PAGE\_SIZE}}.$$

`calculate_order()` searches upward from the order that fits `slub_min_objects` (default derived from `nr_cpu_ids`) to `slub_max_order` (default 3), accepting the first order whose waste is at most $1/$`fract_leftover` of the slab. Larger orders reduce waste for awkward sizes but increase the chance of allocation failure under fragmentation, so the ceiling matters.

The classic bad case is an object just over a power of two. A 520-byte object in a 4 KiB slab yields $n = 7$ and 11% waste; raised to order 1 it yields $n = 15$ and 4.7%. This is why adding one field to a hot structure can cost far more memory than the field's size, and why `pahole` is a standard tool for kernel developers.

7.4 kmalloc

`kmalloc()` is a set of general-purpose caches in power-of-two size classes from 8 to 8192 bytes, plus the intermediate classes 96 and 192 that cover two very common structure sizes. Requests are rounded up to the enclosing class, so worst-case internal fragmentation approaches

$$1 - \frac{2^m + 1}{2^{m+1}} \approx 50\%$$

for a request just over a class boundary. `kmalloc_size_roundup()` exists so callers can ask for the rounded size and use the slack rather than waste it.

Above `KMALLOC_MAX_CACHE_SIZE` (8 KiB), `kmalloc()` falls through to the page allocator, which means large `kmalloc()`s inherit the page allocator's failure modes: they need physically contiguous memory and can fail under fragmentation even when memory is available. `kvmalloc()` is the correct answer for anything large and size-variable — it tries `kmalloc()` with `__GFP_NORETRY` and falls back to `vmalloc()`. Several remote-triggerable denial-of-service bugs have come from a large `kmalloc()` on a size the attacker controls.

There are separate `GFP_DMA` and `-cg` (memcg-accounted) kmalloc arrays, and since 6.6 a **random** set of caches (`CONFIG_RANDOM_KMALLOC_CACHES`) to make heap-spraying exploits harder by making an object's cache placement unpredictable.

7.5 Debugging

SLUB carries substantial optional instrumentation, controlled at boot by `slub_debug=` or per cache: red zones around objects to catch overflows, poison patterns to catch use-after-free, owner tracking recording the allocating and freeing stack for every object, and `CONFIG_SLUB_DEBUG_ON`. `KASAN` builds on the same infrastructure with shadow memory and quarantine, and `KFENCE` provides a low-overhead sampling variant suitable for production.

8 vmalloc

`vmalloc()` allocates virtually contiguous, physically scattered memory: allocate n order-0 pages individually, then map them consecutively into the `vmalloc` region of the kernel address space.

The trade:

- **Advantage.** Immune to external fragmentation. A 16 MiB `vmalloc()` needs 4096 order-0 pages, which almost always exist; the equivalent `kmalloc()` needs an order-12 block, which does not exist at all since `MAX_PAGE_ORDER` is 10.
- **Disadvantage.** Not DMA-capable without scatter-gather, since the physical pages are scattered. Requires page table setup and TLB entries of its own rather than riding the direct map (which is mapped with huge pages, so direct-map memory is nearly TLB-free). Freeing requires a TLB flush and, historically, a global one.

`mm/vmalloc.c` manages the region with a red-black tree of `struct vmap_area` plus a per-CPU `vmap_block` layer (`vm_map_ram()`) for small short-lived mappings. Lazy TLB flushing batches unmaps: freed areas accumulate on a purge list until `lazy_max_pages()` is exceeded, then one flush covers all of them. Since 5.2 `__vmalloc_node_range()` can use huge PMD mappings for large allocations (`VM_ALLOW_HUGE_VMAP`), which matters for the module text and for large hash tables.

Kernel thread stacks are `vmalloc`ed by default (`CONFIG_VMAP_STACK`), which gives every stack a guard page and turns kernel stack overflow from silent memory corruption into a clean fault. `vfree()` may not be called from interrupt context; the deferred path (`vfree_atomic()`) exists for that.

9 The process address space

9.1 mm_struct and VMAs

A process's address space is a `struct mm_struct` holding the page table root (`pgd`), accounting counters, and a set of **virtual memory areas**:

```
struct vm_area_struct {
        unsigned long           vm_start;       /* inclusive */
        unsigned long           vm_end;         /* exclusive */
        struct mm_struct        *vm_mm;
        pgprot_t                vm_page_prot;
        unsigned long           vm_flags;       /* VM_READ, VM_WRITE, ... */
        struct file             *vm_file;
        unsigned long           vm_pgoff;
        const struct vm_operations_struct *vm_ops;
        struct anon_vma         *anon_vma;
        ...
};
```

A VMA is a maximal range with uniform properties: same protections, same backing, same flags. `mmap()` creates them, `munmap()` splits and removes them, `mprotect()` splits and modifies them, and `vma_merge()` recombines adjacent compatible ones so the count does not grow without bound. The count is capped by `sysctl_max_map_count` (default 65530), a limit that Java heaps, Electron applications and Chrome regularly run into.

Crucially, a VMA describes what memory *should* be there, while the page tables describe what *is* there. The gap between the two is where demand paging lives, and the fault handler's job is to close it.

9.2 The maple tree

Until 6.1 the VMAs of an `mm` were held in a red-black tree plus a linked list plus a per-thread one-entry cache. Liam Howlett's **maple tree** (`lib/maple_tree.c`, `Documentation/core-api/maple_tree.rst`) replaced all three.

The maple tree is an RCU-safe, range-indexed B-tree. The advantages over the rbtree it replaced:

- **Cache behaviour.** A B-tree node holds up to 16 slots in a couple of cache lines, so a lookup touches far fewer lines than an rbtree of the same size, where every node is a separate allocation and every step is a dependent load.
- **Ranges are native.** VMAs are intervals; the rbtree stored them by start address and needed augmentation for gap-finding. The maple tree stores ranges directly, so `mmap()`'s unmapped-gap search is a tree property rather than a separate augmented walk.
- **Lockless readers.** RCU-safe lookup is what makes per-VMA locking possible.

`vma_iter_*` and the `VMA_ITERATOR` macro are the modern iteration API; `find_vma()` remains for lookup. Much of `mm/mmap.c` has since been split into `mm/vma.c`.

9.3 Locking

The `mmap_lock` (a read-write semaphore, renamed from `mmap_sem` in 5.8) protects the VMA structures. Historically every page fault took it for reading, making it one of the most contended locks in the kernel on many-threaded workloads: a thousand threads faulting in parallel serialize on the reader count's cache line, and any writer (`mmap()`, `munmap()`, `mprotect()`) blocks all of them.

Per-VMA locks (Suren Baghdasaryan, 6.4) fix the common case. A fault first tries `lock_vma_under_rcu()`, which finds the VMA via the RCU-safe maple tree and takes a per-VMA read lock; only on failure does it fall back to `mmap_lock` and retry. Since faults on distinct VMAs no longer contend, and the overwhelming majority of faults hit a VMA nobody is modifying, this removed a major scalability ceiling for large multithreaded processes.

9.4 Layout and randomization

`mmap()` with no fixed address calls `arch_get_unmapped_area()` (or the `_topdown` variant, the usual choice), which searches the maple tree for a gap. ASLR randomizes the stack, the mmap base, the heap start, and, for PIE binaries, the executable itself; entropy is architecture- and config-dependent (`mmap_rnd_bits`, 28 bits by default on x86-64). `randomize_va_space` selects the level.

9.5 Overcommit

Because pages are allocated on first touch, the kernel can promise more than it has. `sysctl_overcommit_memory` selects the policy, enforced in `__vm_enough_memory()` (`mm/util.c`):

- **0, `OVERCOMMIT_GUESS`** (default). Refuse only obviously absurd requests — roughly, a single allocation larger than RAM plus swap.
- **1, `OVERCOMMIT_ALWAYS`.** Never refuse. Required by workloads that map enormous sparse regions.
- **2, `OVERCOMMIT_NEVER`.** Strict accounting against a commit limit:

$$\texttt{CommitLimit} = \texttt{swap} + \texttt{RAM} \cdot \frac{\texttt{overcommit\_ratio}}{100},$$

with `overcommit_ratio` defaulting to 50, or an absolute `overcommit_kbytes` if set. Non-root allocations are additionally denied the last $1/32$ of the limit, reserving about 3% so that root can log in and repair a machine that has hit the wall.

Mode 2 trades OOM kills for `malloc()` failures. That is the right trade for systems where allocation failure is handled and being killed is not, and the wrong one for the general case, since the default 50% ratio makes a machine refuse allocations while half its RAM sits free. `/proc/meminfo` reports `CommitLimit` and `Committed_AS`.

10 Page fault handling

10.1 The path

A fault begins in architecture code (`arch/x86/mm/fault.c`, `exc_page_fault()`), which reads the faulting address from `CR2` and the error code from the exception frame, disposes of kernel-address faults and `vmalloc` faults, and calls into generic code:

```
handle_mm_fault()
  -> __handle_mm_fault()
       pgd/p4d/pud/pmd walk, allocating tables as needed
       handle_pte_fault()
            do_pte_missing()      /* PTE is empty */
                 do_anonymous_page()      /* no vm_file  */
                 do_fault()               /* file-backed */
                      do_read_fault() / do_cow_fault() / do_shared_fault()
            do_swap_page()        /* PTE holds a swap entry */
            do_wp_page()          /* write to a read-only present PTE */
            do_numa_page()        /* PROT_NONE NUMA hint fault */
```

The return value is a `vm_fault_t` bitmask: `VM_FAULT_MAJOR` if I/O was needed, `VM_FAULT_RETRY` if the handler dropped `mmap_lock` and wants to be re-entered, `VM_FAULT_OOM`, `VM_FAULT_SIGBUS`, and so on. Distinguishing minor faults (page already in memory, just needs a PTE) from major faults (requires I/O) is the basic performance question, and both are counted per task in `/proc/PID/stat`.

10.2 Anonymous faults

A read fault on untouched anonymous memory does not allocate. It maps the shared **zero page**, read-only:

```
if (!(vmf->flags & FAULT_FLAG_WRITE) && !mm_forbids_zeropage(vma->vm_mm)) {
        entry = pte_mkspecial(pfn_pte(my_zero_pfn(vmf->address),
                                      vma->vm_page_prot));
        ...
}
```

A large calloc'd array that is only read costs one physical page in total. The first *write* to any of it takes a write-protect fault and allocates for real.

Write faults allocate a zeroed page, charge it to the memcg, add it to the anon LRU and the rmap, and install a writable PTE. Zeroing is mandatory for security — the page may contain another process's data — and shows up as real cost: `clear_page()` on a 2 MiB THP is not free, and is why THP can increase fault latency even as it reduces fault count.

10.3 Copy-on-write

`fork()` does not copy pages. It copies page tables with every writable private PTE marked read-only in both parent and child, and takes a reference on each page. The first write faults into `do_wp_page()`, which either copies the page or, if this is the only reference, simply makes it writable again ("reuse").

Getting the reuse test right has been genuinely hard. The naive `page_mapcount() == 1` check is wrong in the presence of `get_user_pages()` references held by drivers or in-flight DMA: a COW that copies away from a page that a device is writing into produces silent data loss or, in the other direction, cross-process data leaks. This was CVE-2020-29374 and a family of related bugs, resolved over several releases by tracking pins separately (`folio_maybe_dma_pinned()`, `FOLL_PIN` versus `FOLL_GET`, and the "COW mapcount" rework in 6.x). The current logic lives in `wp_can_reuse_anon_folio()` and is worth reading precisely because the naive version looks obviously correct and is not.

The famous **Dirty COW** race (CVE-2016-5195) was a different bug in the same area: a race between the COW fault path and `madvise(MADV_DONTNEED)` allowed a write to land on the original

read-only page, giving unprivileged write access to read-only files.

10.4 Fault-around

Taking a fault per 4 KiB page to map a file that is already fully cached is wasteful — the expensive part is the trap, not the PTE store. `do_read_fault()` calls `do_fault_around()`, which maps up to `fault_around_bytes` (default 64 KiB, tunable via debugfs) of already-resident neighbouring pages in one go. Startup of a large binary can take a small fraction of the faults it otherwise would.

This is a pure latency-versus-memory trade: mapping pages that are never used costs page table entries and rmap work. The default is conservative for that reason, and the mechanism only maps pages already in the page cache — it never initiates I/O.

10.5 Huge page faults

If the VMA is suitably aligned and sized and THP is enabled, `create_huge_pmd()` installs a 2 MiB PMD-level mapping directly, allocating an order-9 folio. The fallback chain when a huge allocation is unavailable is what determines whether THP helps or hurts: `defer` and `defer+madvise` fall back to 4 KiB immediately and let `khugepaged` fix it later, while `always` with `defrag=always` will synchronously compact, which is where multi-hundred-millisecond fault latencies come from.

11 The page cache

11.1 Structure

The page cache holds file data in memory, indexed per file by offset. The index is an **XArray** (`struct address_space::i_pages`), which since 4.20 replaced the older radix tree API while keeping the same underlying structure: a radix tree with a branching factor of 64 (6 bits per level), tagged entries, and RCU-safe lookup.

```
struct address_space {
        struct inode            *host;
        struct xarray           i_pages;
        atomic_t                nr_thpages;
        unsigned long           nrpages;
        const struct address_space_operations *a_ops;
        unsigned long           flags;
        struct rw_semaphore     invalidate_lock;
        ...
};
```

Tags on the tree (`PAGECACHE_TAG_DIRTY`, `PAGECACHE_TAG_WRITEBACK`, `PAGECACHE_TAG_TOWRITE`) let writeback find dirty pages of a file without scanning it, propagating up the tree so a subtree with no dirty pages is skipped entirely. This is what makes `sync` on a large mostly-clean file cheap.

Slots can also hold **shadow entries** rather than pages: when a page is evicted, a value entry recording *when* it was evicted is left behind. That is the basis of refault detection (Section 15).

The cache is unified with the memory map: `mmap()` of a file installs PTEs pointing at the very same pages that `read()` would copy from, so there is no coherence problem between the two interfaces and no double caching. This was not always true — it is the "unified page cache" that arrived in 2.4 — and it remains one of the more consequential design decisions in the kernel.

11.2 Folios in the page cache

Large folios in the page cache (5.16 onward) let a file be cached in blocks larger than 4 KiB: one descriptor, one refcount, one LRU entry, and one set of rmap operations for what used to be up to 512 of each. For a fast NVMe device the per-page overhead of the old scheme was a measurable fraction of total I/O cost. `filemap_get_folio()`, `folio_size()`, and `mapping_set_large_folios()` are the visible API.

11.3 Readahead

Sequential reads should not pay a fault or an I/O round trip per block. `mm/readahead.c` maintains per-file state:

```
struct file_ra_state {
        pgoff_t start;          /* where the current window begins */
        unsigned int size;      /* window size in pages */
        unsigned int async_size;/* trigger point within the window */
        unsigned int ra_pages;  /* maximum, from the bdi */
        ...
};
```

Two mechanisms combine. **Synchronous readahead** fires on a cache miss that looks sequential and reads a window ahead. **Asynchronous readahead** is triggered by a marker: the page at `start + size - async_size` is tagged `PG_readahead`, and touching it starts the *next* window's I/O while the application is still consuming the current one. Done right, the application never waits.

The window grows multiplicatively while the pattern holds (`get_next_ra_size()`):

$$\texttt{size}_{n+1} = \begin{cases} 4 \cdot \texttt{size}_n, & \texttt{size}_n < \texttt{max}/16 \\ 2 \cdot \texttt{size}_n, & \text{otherwise} \end{cases} \qquad \text{capped at } \texttt{max}.$$

Fast ramp while the window is small, then geometric growth, then a ceiling — `bdi->ra_pages`, default 128 KiB, raised by `blockdev --setra` or per-mount. A detected random pattern collapses the window to zero, and `posix_fadvise()` and `madvise()` let applications state the pattern outright (`POSIX_FADV_SEQUENTIAL`, `_RANDOM`, `_WILLNEED`, `_DONTNEED`), which is usually better than any heuristic.

12 Reverse mapping

12.1 The problem

Forward mapping (virtual to physical) is what page tables do. Reclaim needs the opposite: given a physical page about to be evicted or migrated, find and unmap **every** PTE that references it. Without an answer, a page cannot be evicted, migrated, or compacted.

Early kernels scanned every page table of every process. That is $O(\text{total mapped pages})$ per page reclaimed, and it made reclaim quadratic on large machines. The 2.6-era answer was to store a list of PTEs per page, but the list allocation itself failed under memory pressure — precisely when reclaim is needed.

12.2 Object-based rmap

The solution (Andrea Arcangeli, Hugh Dickins, and others) stores no per-page list. It records, per page, enough to *derive* the mappings from the VMAs that could contain it. `page->mapping` is overloaded by page type:

File pages. `folio->mapping` points to the `address_space`, and `folio->index` gives the offset. Every VMA mapping that file is on the `address_space`'s interval tree `i_mmap`, keyed by file offset. So the mappers of a page are found by querying the interval tree for VMAs covering `folio->index`, and each candidate VMA yields a virtual address by arithmetic:

$$\texttt{addr} = \texttt{vma->vm\_start} + \big(\texttt{index} - \texttt{vma->vm\_pgoff}\big) \cdot \texttt{PAGE\_SIZE}.$$

Anonymous pages. There is no file, so the kernel interposes a `struct anon_vma`, allocated when a VMA first faults in an anonymous page. `folio->mapping` points at it (with a low bit set to distinguish it from an `address_space`), and the `anon_vma` owns an interval tree of the VMAs that may map its pages.

`fork()` complicates this: parent and child share pages, so both VMAs must be reachable. The `anon_vma_chain` links a child's VMA to its parent's `anon_vma` as well as its own, forming a hierarchy. `rmap_walk_anon()` therefore visits potentially several `anon_vma`s. Long chains of forks without exec can make this expensive; `anon_vma_clone()` and the "reuse" heuristics in `anon_vma_fork()` exist to bound it.

The entry points are `rmap_walk()`, `try_to_unmap()` (reclaim), and `try_to_migrate()` (migration and compaction). `folio_referenced()` uses the same walk to collect and clear accessed bits, which is how reclaim learns anything about a page's recent use.

12.3 Cost

An rmap walk is $O(\text{number of VMAs mapping the page})$ plus the interval tree lookups. For a private anonymous page that is one VMA and the walk is trivial. For a page of `libc` mapped by every process on the system it is thousands, which is why heavily shared pages are expensive to reclaim and why reclaim tends to leave them alone. `page_mapcount()` gives the count without the walk and is used as a cheap filter throughout `mm/vmscan.c`.

13 Page replacement: the theory

13.1 The optimal policy and why it is unavailable

Given a cache of N pages and a known reference string, the policy minimizing faults is **Belady's OPT**: evict the page whose next use is furthest in the future. It requires knowledge of the future and is therefore only a benchmark, but it is a useful one — the gap between a real policy and OPT on a traced workload bounds how much a better policy could possibly buy.

Practical policies approximate OPT by assuming the recent past predicts the near future. **LRU** evicts the least recently used page. It is optimal for reference strings with locality and pathological for cyclic access over a working set slightly larger than the cache, where it achieves a 0% hit rate on a pattern that any random policy would partially serve.

13.2 Stack algorithms and Belady's anomaly

A replacement policy is a **stack algorithm** if for all reference strings and all N,

$$\mathcal{C}_N(\sigma) \subseteq \mathcal{C}_{N+1}(\sigma),$$

where $\mathcal{C}_N(\sigma)$ is the cache content after string σ with capacity N. That is, a bigger cache always holds a superset of what a smaller one holds. LRU and OPT are stack algorithms; FIFO is not.

Stack algorithms cannot exhibit **Belady's anomaly**, in which adding memory *increases* the fault count — a real and initially astonishing phenomenon that FIFO does exhibit.

The stack property gives a strong practical tool. Define the **reuse distance** (or LRU stack distance) of a reference as the number of *distinct* pages accessed since that page was last accessed. Then under LRU:

$$\text{a reference hits} \iff \text{reuse distance} < N.$$

So the entire miss-ratio curve over all cache sizes can be obtained from a single pass computing the reuse-distance histogram (Mattson et al., 1970). This is not merely theoretical bookkeeping in Linux: it is exactly what the refault machinery in `mm/workingset.c` measures, and it is the most elegant piece of mathematics in the mm subsystem.

13.3 CLOCK and the accessed bit

True LRU requires updating a global order on every access, which is impossible without hardware support — and hardware provides only a single accessed bit per PTE. **CLOCK** (second-chance) approximates LRU with it: keep pages in a circular list with a hand; on eviction, inspect the page under the hand; if its accessed bit is set, clear it and advance; if clear, evict. A page survives one full revolution of the hand after each reference, so the accessed bit is effectively a one-bit recency timestamp with the revolution period as its resolution.

Linux is a two-list variant of CLOCK, described next.

14 Linux's page replacement

14.1 The two-list scheme

Each node (`struct lruvec`, per-node and per-memcg) keeps five lists:

```
enum lru_list {
        LRU_INACTIVE_ANON, LRU_ACTIVE_ANON,
        LRU_INACTIVE_FILE, LRU_ACTIVE_FILE,
        LRU_UNEVICTABLE,
};
```

The **active/inactive** split is a second-chance mechanism that solves LRU's worst failure mode. Under plain LRU, streaming a large file once evicts the entire working set, because every streamed page enters at the most-recently-used end and pushes everything else out. This is *cache pollution*, and it used to be trivially reproducible with `cat bigfile > /dev/null`.

Linux avoids it with a promotion rule: **new pages enter the inactive list, and are promoted to active only on a second reference while still on the inactive list.** A page referenced exactly once never reaches the active list, so a single-pass stream churns only the inactive list and leaves the working set intact. The inactive list thus functions as an admission filter, not merely as the eviction end of a queue.

`ACTIVE` and `INACTIVE` are separately maintained for anonymous and file pages because they have very different eviction costs — evicting a clean file page costs nothing, evicting an anonymous page requires a swap write.

The kernel targets a rough balance between the file lists, checked by `inactive_is_low()`: the inactive list should hold at least as many pages as the active list for large zones, so there is always enough inactive space for the second-reference test to be meaningful. If the inactive list is too

small, pages get evicted before they have a chance to be referenced twice and the admission filter stops working.

`mark_page_accessed()` (now `folio_mark_accessed()`) implements the promotion; `shrink_active_list()` demotes back, consulting `folio_referenced()` to give a page a last chance.

14.2 Refault distance: measuring what was lost

The two-list scheme has a blind spot. Once a page is evicted, the kernel knows nothing more about it. If the workload's working set is slightly larger than memory, file pages are evicted and immediately read back — **thrashing** — while the anonymous active list sits untouched, and nothing in the LRU state reveals that the file cache is too small rather than merely cold.

Johannes Weiner's solution (3.15, `mm/workingset.c`) is to keep measuring after eviction, using the stack-distance identity above.

Maintain a per-lruvec counter of evictions. When a page is evicted, store the counter's current value E in the page cache slot the page vacated — a **shadow entry**, packed with the memcg id and node id into a single XArray value. When that page faults back in at counter value R, compute

$$d = R - E,$$

the number of pages evicted from this list in the interim. Since eviction is from the inactive list's tail, d is exactly the number of *distinct* pages that pushed this one out — its reuse distance beyond the inactive list. So the page would have been a hit if the list had been larger by d. Compare against the size of the memory it is competing with:

$$\text{activate on refault} \iff d \leq \texttt{workingset\_size},$$

with `workingset_size` the active file list plus, when swap is available, the anonymous lists — the memory that could in principle have been given up instead.

The consequences are what make this valuable. A page that refaults within the distance is promoted **straight to the active list**, skipping the second-reference requirement, because the evidence that it is part of the working set has already been gathered. And the event is counted (`WORKINGSET_REFAULT`, `WORKINGSET_ACTIVATE`, `WORKINGSET_RESTORE` in `/proc/vmstat` and `memory.stat`), which gives userspace a direct thrashing signal. This counter is what PSI memory pressure is built on.

Shadow entries consume memory, so they are themselves reclaimable: `workingset_shadow_shrinker` drops them under pressure, and the eviction counter's limited bit width means very old shadows are ambiguous and treated as misses.

14.3 MGLRU

Multi-generational LRU (Yu Zhao, merged in 6.1, `CONFIG_LRU_GEN`) is a rework of the aging side. The complaints about the classic scheme: `rmap` walks to gather referenced bits are expensive and are done page-by-page during reclaim, when latency matters most; and two lists give only two levels of recency, which is a coarse signal.

MGLRU replaces the two lists with up to `MAX_NR_GENS` (4) **generations** per type. Two operations:

- **Aging** creates a new `max_seq` generation and ages pages into it, scanning page tables *directly* rather than via rmap. Walking a page table sequentially reads many accessed bits per cache

line touched; rmap-walking n pages does n independent tree lookups. For dense mappings this is dramatically cheaper, and MGLRU uses per-VMA and per-PMD bloom-filter-like hints (`lru_gen_mm_walk`) to skip regions with no young pages.
- **Eviction** consumes the oldest generation (`min_seq`).

Within a generation, pages are further sorted into up to `MAX_NR_TIERS` (4) **tiers** by access count, with tier index $\lfloor \log_2(\text{refs} + 1) \rfloor$ — a logarithmic bucketing that costs two bits in the folio flags rather than a counter.

Tier and type selection use a **proportional controller** (`read_ctrl_pos()`, `positive_ctrl_err()`) comparing the refault rate of each candidate against the cost of reclaiming it, weighted by a gain factor. Feedback rather than the fixed heuristics of the classic path.

MGLRU coexists with the classic code; `/sys/kernel/mm/lru_gen/enabled` selects it, and `/sys/kernel/debug/lru_gen` exposes the generation structure. It is default-on in Android and ChromeOS and increasingly elsewhere.

15 Reclaim mechanics

15.1 Who reclaims, and when

- **`kswapd`** — one kernel thread per node, woken when a zone drops below its low watermark, reclaiming until it is above high. Asynchronous, so allocators do not stall.
- **Direct reclaim** — an allocating task that finds the zone below `min` does the work itself, in its own context, before its allocation can proceed. This is a latency event and shows up as `allocstall` in `/proc/vmstat` and as memory pressure in PSI.
- **`kcompactd`**, **memcg reclaim**, and **proactive reclaim** (`memory.reclaim`) round out the set.

15.2 The priority loop

`shrink_node()` scans a fraction of each LRU determined by a **priority**, counting down from `DEF_PRIORITY` (12) to 0:

$$\texttt{nr\_to\_scan} = \frac{\texttt{lruvec\_size}}{2^{\texttt{priority}}}.$$

The first pass scans $1/4096$ of the list. If that produces enough free pages, reclaim stops having touched almost nothing. If not, priority decreases and the scan doubles, so cost escalates geometrically with need, and by priority 0 the entire list is scanned. Reaching low priorities is itself the signal for further escalation — it enables writeback of dirty pages from reclaim context, unmapping of mapped pages, and eventually the OOM killer.

This structure is what keeps reclaim cheap in the common case. Most reclaim events are satisfied at priority 12 by dropping a handful of clean page cache pages.

15.3 Anon versus file balance

`get_scan_count()` divides the scan between anonymous and file lists using both a policy knob and measured cost. The kernel tracks `anon_cost` and `file_cost` — the recent reclaim effort per list, incremented by rotations, which represent wasted work. Then:

$$\texttt{ap} = \texttt{swappiness} \cdot \frac{\texttt{total\_cost} + 1}{\texttt{anon\_cost} + 1}, \qquad \texttt{fp} = (200 - \texttt{swappiness}) \cdot \frac{\texttt{total\_cost} + 1}{\texttt{file\_cost} + 1},$$

and the scan of each list is apportioned as $\texttt{ap}/(\texttt{ap} + \texttt{fp})$ and $\texttt{fp}/(\texttt{ap} + \texttt{fp})$.

`swappiness` (default 60, range 0–200) is therefore not "how much to swap" but a **prior** on the relative value of anonymous versus file memory, which the measured costs then correct. Two consequences that surprise people:

- `swappiness=0` does not disable swapping. It removes the prior in favour of anonymous memory; the kernel will still swap rather than OOM.
- The maximum is 200, not 100. Values above 100 bias toward anonymous reclaim, which is the right setting when swap is as fast as the filesystem — zram, or NVMe swap against network storage.

Additional gates: if there is no swap or no swap space free, anonymous scanning is skipped entirely; if file pages alone are enough to meet the watermark, `sc->cache_trim_mode` skips anonymous scanning; and `vm.watermark_boost_factor` temporarily raises watermarks after a fragmentation event to reclaim proactively and let compaction work.

15.4 The scan itself

`shrink_inactive_list()` isolates a batch from the tail, then `shrink_folio_list()` decides each page's fate:

1. Referenced since last check? Rotate to the head, or activate.
2. Dirty? If reclaim is at low priority and writeback is congested, this triggers throttling rather than a synchronous write — writing back from reclaim context produces random I/O and is a known pathology, so the kernel prefers to wait for the flusher threads.
3. Under writeback? Wait or skip.
4. Mapped? `try_to_unmap()` via rmap.
5. Anonymous? Allocate a swap slot and queue the write.
6. Clean file page? Free immediately. This is the cheap case, and the one reclaim hopes for.

`reclaim_throttle()` (`VMSCAN_THROTTLE_WRITEBACK` and friends) replaced the older unconditional `congestion_wait()` sleeps, which used to cause multi-hundred-millisecond stalls even when there was nothing to wait for.

15.5 Shrinkers

Not all reclaimable memory is on an LRU. Dentries, inodes, filesystem metadata caches, and dozens of subsystem caches register a **shrinker**:

```
struct shrinker {
        unsigned long (*count_objects)(struct shrinker *, struct shrink_control *);
        unsigned long (*scan_objects)(struct shrinker *, struct shrink_control *);
        int seeks;      /* cost to rebuild an object, in seek units */
        long batch;
        ...
};
```

`do_shrink_slab()` converts the reclaim priority into a scan count, scaled by the declared rebuild cost:

$$\Delta = \frac{4 \cdot \texttt{freeable}}{2^{\texttt{priority}} \cdot \texttt{seeks}},$$

with `DEFAULT_SEEKS = 2`, giving $\Delta = 2\,\texttt{freeable}/2^{\texttt{priority}}$ — the same geometric escalation as the LRU scan. A cache that is expensive to rebuild declares a higher `seeks` and is scanned proportionally less. Fractional remainders are carried in `nr_deferred` so that repeated small requests eventually accumulate into real work.

Shrinkers are memcg-aware and NUMA-aware when they set `SHRINKER_MEMCG_AWARE` / `SHRINKER_NUMA_AWARE`; the per-memcg shrinker bitmap makes it cheap to skip shrinkers with nothing to free in a given cgroup.

16 Swap

16.1 Slots and the swap cache

A swap area is an array of page-sized slots described by `struct swap_info_struct`, with a `swap_map` array of per-slot reference counts. Slot allocation (`get_swap_page()`) uses per-CPU clusters to keep a task's pages contiguous on the device, which mattered enormously for rotational media and still matters for read amplification on SSDs.

The **swap cache** is a page cache keyed by swap entry rather than by file offset. It exists to solve a concurrency problem: if two processes share an anonymous page that has been swapped out, both may fault on it simultaneously, and without a common lookup point both would read it in and produce two copies of what must remain one page. The swap cache gives them a place to meet.

A swapped-out page's PTE holds a `swp_entry_t`; `do_swap_page()` decodes it, looks in the swap cache, and reads from the device on a miss (`swap_readahead()` provides cluster and VMA-based readahead policies).

16.2 Writeback of anonymous memory

Anonymous reclaim is strictly more expensive than clean file reclaim: the page must be written before its frame can be reused, so the reclaim path has a synchronous dependency on device latency. This asymmetry, not any dislike of swap as such, is why the kernel defaults to preferring file reclaim.

The common advice to disable swap entirely is usually wrong. Without swap the kernel cannot evict *any* anonymous page, so all reclaim pressure lands on the page cache; a process with a large cold heap will force out the file cache the whole system needs, and the machine gets slower rather than faster. It also loses the ability to reclaim never-touched-again initialization allocations, which every long-lived process accumulates.

zram (a compressed RAM block device used as swap) and **zswap** (a compressed writeback cache in front of a real swap device) change the economics completely, since the "write" is a compression rather than an I/O. Typical compression ratios of 2–3x on anonymous memory mean swapping to zram costs microseconds instead of milliseconds, which is why `swappiness` above 100 makes sense there. `zsmalloc` is the backing allocator, designed for storing variable-size compressed pages with low fragmentation.

17 Writeback and dirty throttling

17.1 The problem

A process writing to a file dirties page cache faster than any device can drain. Left unchecked, all of memory becomes dirty, reclaim finds nothing clean to free, and the machine stalls in a large uncontrolled burst. The kernel must therefore rate-limit dirtying to match device throughput — a **control problem**, and `mm/page-writeback.c` is a genuine controller, largely the work of Fengguang Wu (3.1–3.2).

17.2 Limits

Two thresholds, expressed as percentages of *available* memory (free plus reclaimable file pages), or as absolute byte counts:

Tunable	Default	Effect
`dirty_background_ratio`	10%	Flusher threads start writing back
`dirty_ratio`	20%	Writers are throttled
`dirty_expire_centisecs`	3000	Age at which a page must be written
`dirty_writeback_centisecs`	500	Flusher wakeup interval

Below the background threshold, nothing happens and writes are pure memory stores. Between the two, flusher threads (per backing device, `struct bdi_writeback`) write in the background. Above `dirty_ratio`, `balance_dirty_pages()` makes writers sleep.

17.3 The control law

Naive throttling — block hard at the limit — produces oscillation: everything runs at full speed until the wall, then everything stops. The kernel instead uses a smooth feedback loop with a setpoint in the middle of the operating range:

$$\texttt{freerun} = \frac{\texttt{thresh} + \texttt{bg\_thresh}}{2}, \qquad \texttt{setpoint} = \frac{\texttt{freerun} + \texttt{limit}}{2},$$

and computes a dimensionless correction, the **position ratio**, as a cubic in the normalized error:

$$\texttt{pos\_ratio}(x) \;=\; 1 + \left(\frac{\texttt{setpoint} - x}{\texttt{limit} - \texttt{setpoint}} \right)^3, \qquad x = \text{current dirty pages}.$$

The cubic is chosen to satisfy four constraints simultaneously:

$$\texttt{pos\_ratio}(\texttt{freerun}) = 2, \qquad \texttt{pos\_ratio}(\texttt{setpoint}) = 1,$$

$$\texttt{pos\_ratio}(\texttt{limit}) = 0, \qquad \frac{\mathrm{d}\,\texttt{pos\_ratio}}{\mathrm{d}x} \le 0.$$

Negative feedback with a hard zero at the limit, unity gain at the setpoint, and — the reason for the odd power specifically — a derivative that is *small near the setpoint and large near the extremes*. The system responds gently to small errors, so it does not oscillate around the operating point, and aggressively to large ones, so it cannot run away. A linear law gives up the first property; a steeper law gives up stability.

17.4 Bandwidth estimation and the rate limit

The controller needs to know how fast the device actually is, which it measures rather than assumes. Every `BANDWIDTH_INTERVAL` (200 ms), `__wb_update_bandwidth()` computes observed throughput and folds it into an exponentially weighted average with a time constant of a few seconds:

$$\texttt{write\_bw} \leftarrow \texttt{write\_bw} + \frac{\texttt{observed} - \texttt{write\_bw}}{8}.$$

The per-device `dirty_ratelimit` is then updated toward the rate that would hold dirty pages at the setpoint:

$$\texttt{balanced\_ratelimit} = \texttt{dirty\_ratelimit} \cdot \frac{\texttt{write\_bw}}{\texttt{dirty\_rate}},$$

with the update step-limited to avoid overshoot, and each task's allowance scaled by its position:

$$\texttt{task\_ratelimit} = \texttt{dirty\_ratelimit} \times \texttt{pos\_ratio}.$$

Finally, a task that has dirtied n pages sleeps for

$$\texttt{pause} = \frac{n}{\texttt{task\_ratelimit}},$$

capped at `MAX_PAUSE` (200 ms) and applied in small increments so that a writer is smoothly slowed rather than periodically frozen. The design goal stated in the source is that a dirtying task should see steady, small pauses — ideally under 10 ms — rather than the multi-second stalls the pre-3.x code produced.

17.5 Per-device and per-cgroup shares

With several devices of different speeds, a single global limit lets a slow device's backlog throttle writers to a fast one. Each `bdi` therefore gets a share of the global limit proportional to its recent share of writeback (`wb_dirty_limits()`), with its own position ratio, so a USB stick cannot stall writers to NVMe. Cgroup writeback (`CONFIG_CGROUP_WRITEBACK`, 4.2) extends the same split to memcgs, giving each its own `bdi_writeback` and its own share — without which memory cgroup limits and I/O cgroup limits interact incoherently, since the process that dirties a page and the flusher that writes it are different tasks.

18 Huge pages

18.1 THP

Transparent huge pages back a suitably aligned 2 MiB region with one PMD-level mapping. The benefit is TLB reach (Section 3.4); the costs are real and often understated:

- **Allocation latency.** An order-9 folio may require compaction. `defrag=always` can inject hundreds of milliseconds into a page fault.
- **Internal fragmentation.** A 2 MiB mapping for a sparsely touched region wastes up to 2 MiB per fault. Redis and similar workloads have historically seen large RSS inflation from THP, which is why several databases recommend disabling it.
- **Copy and clear cost.** Faulting a THP zeroes 2 MiB; COW copies 2 MiB.

`/sys/kernel/mm/transparent_hugepage/enabled` takes `always`, `madvise`, or `never`; `defrag` separately controls how hard allocation tries. `madvise` is the sane default for general-purpose systems: applications that benefit ask via `madvise(MADV_HUGEPAGE)`, and nothing else pays.

`khugepaged` is the background collapse thread. It scans for regions of 4 KiB pages that could become a THP, allocates a huge folio, copies, and installs the PMD — so a process can get huge

pages without fault-time latency. `mm/khugepaged.c`; scan rate is tunable and the defaults are conservative.

`split_huge_page()` is the reverse, needed whenever a huge mapping must become heterogeneous: `mprotect()` of a sub-range, partial `munmap()`, reclaim of part of it, or migration.

Multi-size THP (mTHP, 6.8 onward) generalizes this to intermediate orders — 16 KiB, 64 KiB, and other sub-PMD sizes — controlled per size under `/sys/kernel/mm/transparent_hugepage/hugepages-*kB/`. Intermediate sizes capture much of the TLB and per-page-overhead benefit at a small fraction of the allocation difficulty and internal waste, and on arm64 they additionally enable hardware contiguous-PTE hints. This is arguably the most significant practical mm change of the 6.x series.

18.2 hugetlbfs

An older, entirely separate mechanism: a boot- or runtime-reserved pool of huge pages (`vm.nr_hugepages`), exposed through a pseudo-filesystem and `MAP_HUGETLB`. Pages come from the pool, never from the buddy allocator, and are never swapped, split, or reclaimed.

The trade against THP is predictability versus flexibility. hugetlb memory is guaranteed, never faults unexpectedly, and supports 1 GiB pages, but it is reserved up front whether used or not, and it is invisible to the normal reclaim and accounting machinery. Databases and hypervisors that want deterministic behaviour use it; general-purpose systems use THP.

`hugetlb_free_vmemmap` (5.14) reclaims most of the `struct page` array for hugetlb pages, since 2^9 or 2^{18} identical tail-page descriptors are compressible to one shared page — recovering roughly 1.5% of all hugetlb-backed memory.

19 NUMA

On a multi-socket machine, memory attached to a remote node costs perhaps 1.5–2.2x the local access latency and shares limited interconnect bandwidth. The kernel's job is to keep a task's pages on the node running it.

Allocation policy (`mm/mempolicy.c`) is set per process or per VMA via `set_mempolicy()`, `mbind()`, and `numactl`: `MPOL_DEFAULT` (local node), `MPOL_BIND`, `MPOL_INTERLEAVE`, `MPOL_PREFERRED`, and `MPOL_WEIGHTED_INTERLEAVE` (6.9, for tiered memory where nodes have different bandwidth). Zonelist order does the rest: local zones first, remote by SLIT distance.

Automatic NUMA balancing (`/proc/sys/kernel/numa_balancing`) corrects placement at runtime. It periodically marks a sample of a task's pages `PROT_NONE` so the next access takes a minor fault; `do_numa_page()` records which node touched the page, and over time the kernel either migrates the page to the task or the task to the page. `task_numa_fault()` and `numa_group` handle the case of several threads sharing pages, which must be placed together. The scan rate adapts: converged tasks are sampled less.

Tiered memory (5.15 onward) generalizes NUMA to nodes of different speeds — CXL-attached memory, persistent memory. Cold pages are **demoted** to a slower node instead of being swapped, and hot pages are **promoted** back. `WMARK_PROMO`, `node_reclaim_mode`, and the `demotion_target` machinery implement it.

20 Cgroups and pressure

20.1 memcg

`mm/memcontrol.c` accounts pages to cgroups and enforces limits. Cgroup v2 exposes:

File	Semantics
`memory.min`	Hard protection. Never reclaimed, even under global pressure.
`memory.low`	Best-effort protection. Reclaimed only if no unprotected memory remains.
`memory.high`	Throttling threshold. Not a hard limit.
`memory.max`	Hard limit. Exceeding it triggers reclaim, then OOM.
`memory.swap.max`, `memory.zswap.max`	Swap limits
`memory.pressure`	PSI for this cgroup
`memory.reclaim`	Write to reclaim proactively

Protection is **propagated proportionally** down the hierarchy (`mem_cgroup_calculate_protection()`). A child's effective protection is its parent's effective protection distributed among siblings in proportion to each one's own claim:

$$E_{\text{child}} = E_{\text{parent}} \cdot \frac{\min(\texttt{usage}_{\text{child}}, \texttt{low}_{\text{child}})}{\sum_{s \in \text{siblings}} \min(\texttt{usage}_s, \texttt{low}_s)}.$$

This makes the setting composable: a parent cannot be circumvented by over-claiming children, and unused protection flows to siblings that can use it.

The distinction between `memory.high` and `memory.max` is the most useful thing in the interface. `max` is a wall: hit it and you are OOM-killed. `high` is a brake: exceed it and the allocating task is made to sleep, with the delay growing **quadratically** in the relative overage (`calculate_high_delay()`):

$$\texttt{penalty} \propto \left(\frac{\texttt{usage} - \texttt{high}}{\texttt{high}}\right)^2,$$

capped at `MEMCG_MAX_HIGH_DELAY_JIFFIES` (2 seconds per allocation). A cgroup slightly over its target is barely slowed; one far over is nearly stopped. This lets a container be held near a target without the cliff, which is why `memory.high` plus a generous `memory.max` is the recommended configuration rather than `max` alone.

20.2 PSI

Pressure Stall Information (Johannes Weiner, 4.20, `kernel/sched/psi.c`) answers the question load average never could: **how much time is lost to resource contention?** It reports, for CPU, memory, and I/O:

- **`some`** — at least one runnable task is stalled on the resource.
- **`full`** — all non-idle tasks are stalled, so the resource is producing no work at all.

`full` is the important one for memory. It means the machine is doing nothing but reclaiming and refaulting.

Values are percentages of wall time, averaged over 10 s, 60 s and 300 s windows with an exponentially weighted average sampled every 2 s. The decay factors are precomputed fixed-point values of $e^{-\Delta t/\tau}$:

$$e_\tau = e^{-2/\tau} \cdot 2^{11}, \qquad e_{10} = 1677, \quad e_{60} = 1981, \quad e_{300} = 2034,$$

with the update

$$\texttt{avg} \leftarrow \frac{\texttt{avg} \cdot e_\tau + \texttt{pct} \cdot (2^{11} - e_\tau)}{2^{11}}.$$

Check: $e^{-0.2} = 0.8187$, and $0.8187 \times 2048 = 1677$.

The memory numerator is built largely on the refault accounting of Section 15.2 — time spent faulting back pages that were recently evicted is, by construction, time lost to insufficient memory. Read `/proc/pressure/memory` or `memory.pressure`; poll with triggers for userspace OOM daemons such as `oomd` and `systemd-oomd`, which can act on pressure long before the kernel's own OOM killer would.

21 The out-of-memory killer

When reclaim cannot make progress and an allocation cannot fail, something must be killed. `mm/oom_kill.c` selects a victim by **badness score** (`oom_badness()`):

$$\texttt{badness} = \texttt{RSS} + \texttt{swap\_entries} + \frac{\texttt{pgtable\_bytes}}{\texttt{PAGE\_SIZE}} + \texttt{oom\_score\_adj} \cdot \frac{\texttt{totalpages}}{1000}.$$

The first three terms are the pages that would actually be recovered by killing the task — resident memory, swap slots, and the page tables that describe them, which for a process with a large sparse mapping can be substantial and were historically ignored.

The fourth term is the administrator's thumb on the scale. `/proc/PID/oom_score_adj` ranges over $[-1000, 1000]$ and is expressed in **thousandths of total memory**, so setting it to $+100$ is equivalent to adding 10% of the machine's RAM to the score, and the scale is meaningful across machines of different sizes. The special value -1000 (`OOM_SCORE_ADJ_MIN`) makes a task ineligible entirely.

Notable properties:

- The score is **per process**, but the kill is per `mm`, and `oom_kill_process()` kills all tasks sharing the victim's `mm`.
- The heuristic is deliberately crude: it targets the largest consumer, which is usually but not always the culprit. There is no attempt to identify which process is *responsible*.
- `panic_on_oom` and `oom_kill_allocating_task` change the policy wholesale.
- `oom_reaper` (4.6) is a kernel thread that asynchronously reaps the victim's anonymous memory with `MADV_DONTNEED` semantics, so the kernel recovers memory even when the victim is blocked and cannot exit. Before it, a victim stuck in uninterruptible sleep could deadlock the whole machine.
- Memcg OOM is separate and applies the same scoring within the cgroup; `memory.oom.group` kills the whole cgroup as a unit, which is usually what a container wants.

In practice the kernel OOM killer engages far too late to be useful for interactive systems — by the time it fires, the machine has usually been thrashing for many seconds. PSI-driven userspace daemons acting on the `full` metric are the modern answer.

22 Observability

```
# System-wide summary and per-zone detail
cat /proc/meminfo
cat /proc/zoneinfo
cat /proc/buddyinfo          # free blocks per order per zone
cat /proc/pagetypeinfo       # free blocks per order per migratetype
cat /proc/slabinfo           # or: slabtop
cat /proc/vmstat             # every counter: pgfault, pgmajfault,
                             # pgscan_*, pgsteal_*, allocstall_*,
                             # compact_*, workingset_*, thp_*

# Per process
cat /proc/PID/status         # VmRSS, VmSwap, VmPTE, RssAnon, RssFile, RssShmem
cat /proc/PID/smaps_rollup   # PSS -- the only fair way to attribute shared memory
cat /proc/PID/maps           # the VMA list
cat /proc/PID/oom_score_adj

# Pressure
cat /proc/pressure/memory
cat /sys/fs/cgroup/<cg>/memory.stat
cat /sys/fs/cgroup/<cg>/memory.pressure

# Tunables
sysctl vm.swappiness vm.min_free_kbytes vm.watermark_scale_factor \
       vm.dirty_ratio vm.dirty_background_ratio vm.overcommit_memory

# Tracing
perf trace -e 'kmem:*'
perf record -e page-faults -e major-faults
trace-cmd record -e vmscan -e compaction -e writeback -e kmem
cat /sys/kernel/debug/tracing/events/vmscan/
```

PSS deserves emphasis. `RSS` double-counts shared pages: sum the `RSS` of every process and you will exceed physical memory, sometimes by a lot. **Proportional set size** divides each shared page by the number of processes mapping it, so PSS *does* sum correctly and is the right metric for "who is using the memory." `smaps_rollup` computes it cheaply; the older per-VMA `smaps` is expensive on large address spaces.

DAMON (`CONFIG_DAMON`, 5.15) is worth knowing about: a data access monitor that samples regions rather than pages, giving access-frequency profiles at bounded overhead regardless of address space size, plus `DAMOS` schemes that can act on what it finds (proactively reclaim cold regions, apply `MADV_HUGEPAGE` to hot ones).

23 Complexity summary

Operation	Cost	Notes
Address translation (TLB hit)	$O(1)$	Hardware
Page walk (TLB miss)	$O(\text{levels}) = 4$ or 5	Each level a potential cache miss
`pfn_to_page`	$O(1)$	Arithmetic under `SPARSEMEM_VMEMMAP`
Buddy alloc / free	$O(\log \texttt{MAX\_PAGE\_ORDER})$	≤ 10 steps; order-0 usually hits the pcp list

Operation	Cost	Notes
SLUB alloc / free (fast)	$O(1)$	Lockless `cmpxchg_double`
VMA lookup	$O(\log n)$	Maple tree
Page cache lookup	$O(\log_{64} n)$	XArray
rmap walk	$O(\text{VMAs mapping the page})$	Plus interval tree lookups
LRU scan at priority p	$O(\texttt{size}/2^p)$	Geometric escalation
TLB shootdown	$O(\text{CPUs in mm\_cpumask})$	IPI broadcast; batched
Compaction	$O(\text{zone size})$ per pass	Two scanners meeting

24 Reading path

The mm subsystem does not have a single entry point, so pick a thread and follow it end to end rather than reading files top to bottom.

1. `Documentation/mm/` — especially `physical_memory.rst`, `page_owner.rst`, and the design notes on THP and MGLRU.
2. **Types first.** `include/linux/mm_types.h` (`struct page`, `folio`, `mm_struct`, `vm_area_struct`), then `include/linux/mmzone.h` (`zone`, `pglist_data`, `lruvec`).
3. **The allocation thread.** `alloc_pages()` through `get_page_from_freelist()` and `__alloc_pages_slowpath()` in `mm/page_alloc.c`, then `zone_watermark_ok()` and `__setup_per_zone_wmarks()`.
4. **The fault thread.** `arch/x86/mm/fault.c::exc_page_fault()` into `handle_mm_fault()`, then `handle_pte_fault()` and each of the `do_*` handlers in `mm/memory.c`. This is the best single file to understand deeply.
5. **The reclaim thread.** `mm/vmscan.c::shrink_node()` down through `get_scan_count()`, `shrink_inactive_list()` and `shrink_folio_list()`; then `mm/workingset.c` in full — it is short and it is the most elegant code in the subsystem.
6. **The writeback thread.** The long comment block above `wb_position_ratio()` in `mm/page-writeback.c` derives the control law before implementing it.
7. `mm/slub.c` fast paths, `mm/rmap.c::rmap_walk()`, and `mm/compaction.c` as needed.

24.1 Books

- Gorman, *Understanding the Linux Virtual Memory Manager* (2004). Describes 2.4/2.6 and is thus historically dated, but the explanations of the buddy allocator, slab, and rmap remain the clearest available and the structures are recognizably the same.
- Love, *Linux Kernel Development*, and Bovet and Cesati, *Understanding the Linux Kernel* — both dated, both still useful for orientation.

24.2 Papers

- Knowlton, *A Fast Storage Allocator* (1965) — the buddy system.
- Bonwick, *The Slab Allocator: An Object-Caching Kernel Memory Allocator* (USENIX 1994).
- Belady, *A Study of Replacement Algorithms for a Virtual-Storage Computer* (1966) — OPT and the anomaly.
- Mattson, Gecsei, Slutz and Traiger, *Evaluation Techniques for Storage Hierarchies* (1970) — stack algorithms and reuse distance, the theory behind `workingset.c`.
- Denning, *The Working Set Model for Program Behavior* (1968).
- Gorman and Healy, *Supporting Superpage Allocation Without Additional Hardware Support* (ISMM 2008) — the antifragmentation design.

24.3 LWN

Search for: “The multi-generational LRU”, “Refault distance”, “No more second-guessing the page allocator”, “The maple tree”, “Folios”, “A memory-management thicket”, “Per-VMA locks”, “Multi-size THP”, “The many faces of dirty page throttling”, “PSI: pressure stall information”. LWN’s memory-management coverage is unusually good and is often the only prose explanation of a mechanism that exists outside the source comments.